

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501
NAME: M.NITHYASHREENARAYANI
REG.NO: 192521416

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find	BL6 – Create	5

	names of students who are enrolled in all courses offered.		
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

I M.NITHYASHREENARAYANI / 192521416 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

DATABASE MANAGEMENT SYSTEMS

Model Answer Key — Assessment

Total Marks: 50 | 10 Questions × 5 Marks

Common Sample Schemas Used Throughout This Answer Key

EMPLOYEE

Column	Data Type	Constraint
EmpID	INT	Primary Key
EmpName	VARCHAR(50)	Not Null
Salary	DECIMAL(10,2)	Not Null
DeptID	INT	Foreign Key → Department(DeptID)

DEPARTMENT

Column	Data Type	Constraint
DeptID	INT	Primary Key
DeptName	VARCHAR(50)	Not Null, Unique
Location	VARCHAR(50)	—

Q1. Relational Algebra Operations on Employee & Department [5 Marks]

Using the EMPLOYEE(EmpID, EmpName, Salary, DeptID) and DEPARTMENT(DeptID, DeptName, Location) relations shown above, the six fundamental relational algebra operations are applied below.

(a) Selection (σ) — Employees with Salary > 50000

$\sigma \text{ Salary} > 50000 (\text{EMPLOYEE})$

(b) Projection (π) — Names and Salaries only

$\pi \text{ EmpName, Salary} (\text{EMPLOYEE})$

(c) Union ($\cup$) — EmpIDs from IT $\cup$ EmpIDs from HR

$\pi \text{ EmpID} (\sigma \text{ DeptID} = 'D01' (\text{EMPLOYEE})) \cup \pi \text{ EmpID} (\sigma \text{ DeptID} = 'D02' (\text{EMPLOYEE}))$

(d) Set Difference ($-$) — Employees in D01 but not in D02

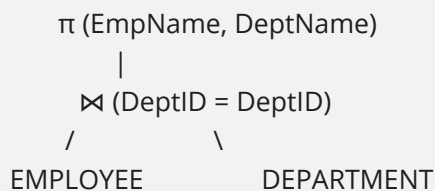
$\pi \text{ EmpID} (\sigma \text{ DeptID} = 'D01' (\text{EMPLOYEE})) - \pi \text{ EmpID} (\sigma \text{ DeptID} = 'D02' (\text{EMPLOYEE}))$

(e) Cartesian Product ($\times$)

EMPLOYEE $\times$ DEPARTMENT
 (produces |EMPLOYEE| $\times$ |DEPARTMENT| tuples, all attribute combinations)

(f) Natural Join ($\bowtie$) — Employee name with Department name

π EmpName, DeptName (EMPLOYEE $\bowtie$ DEPARTMENT)
 (join condition: EMPLOYEE.DeptID = DEPARTMENT.DeptID)

Expression Tree Diagram — for query (f)**Q2. SQL DDL — STUDENT and DEPARTMENT Tables [5 Marks]**

The DEPARTMENT table is created first since STUDENT references it via a foreign key.

```

CREATE TABLE DEPARTMENT (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE STUDENT (
  SID INT PRIMARY KEY,
  Name VARCHAR(50) NOT NULL,
  DOB DATE NOT NULL CHECK (DOB <= CURRENT_DATE),
  DeptID INT,
  CONSTRAINT fk_student_dept
  FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
  ON DELETE SET NULL
  ON UPDATE CASCADE
);
  
```

Constraints applied

- PRIMARY KEY on DeptID and SID — enforces entity integrity (uniqueness, not null).
- NOT NULL on Name and DOB — mandatory attributes.
- UNIQUE on DeptName — no two departments share a name.
- CHECK on DOB — disallows a future date of birth.
- FOREIGN KEY on DeptID — enforces referential integrity between STUDENT and DEPARTMENT, with ON DELETE SET NULL / ON UPDATE CASCADE policies.

Q3. GROUP BY, HAVING, Aggregate Functions on Sales Data [5 Marks]**SALES**

Column	Data Type	Constraint
SaleID	INT	Primary Key
ProductID	INT	Foreign Key
SalesPerson	VARCHAR(50)	Not Null
Region	VARCHAR(30)	Not Null
Amount	DECIMAL(10,2)	Not Null
SaleDate	DATE	Not Null

(a) Total, average, and count of sales per region

```

SELECT Region,
       COUNT(*) AS TotalSales,
       SUM(Amount) AS TotalRevenue,
       AVG(Amount) AS AvgSale,
       MAX(Amount) AS HighestSale,
       MIN(Amount) AS LowestSale
FROM Sales
GROUP BY Region;

```

(b) Regions whose total revenue exceeds 100000

```

SELECT Region, SUM(Amount) AS TotalRevenue
FROM Sales
GROUP BY Region
HAVING SUM(Amount) > 100000
ORDER BY TotalRevenue DESC;

```

(c) Salespersons with more than 10 sales and average sale above 5000

```

SELECT SalesPerson, COUNT(*) AS NumSales, AVG(Amount) AS AvgAmount
FROM Sales
GROUP BY SalesPerson
HAVING COUNT(*) > 10 AND AVG(Amount) > 5000;

```

Note: GROUP BY partitions rows into groups sharing the same value; aggregate functions then act per group; HAVING filters those already-formed groups (unlike WHERE, which filters rows before grouping).

Q4. Correlated Sub-query — Salary Above Department Average [5 Marks]

```

SELECT E1.EmpID, E1.EmpName, E1.Salary, E1.DeptID
FROM Employee E1

```

```
WHERE E1.Salary > (
    SELECT AVG(E2.Salary)
    FROM Employee E2
    WHERE E2.DeptID = E1.DeptID -- correlation with outer query
);
```

This is correlated because the inner query references E1.DeptID from the outer query — the average is recomputed once for every outer row, rather than being evaluated a single time as with a plain sub-query.

Q5. All JOIN Types — Employee & Project [5 Marks]

PROJECT

Column	Data Type	Constraint
ProjectID	INT	Primary Key
ProjectName	VARCHAR(50)	Not Null
EmpID	INT	Foreign Key → Employee(EmpID)

(a) INNER JOIN — employees who are assigned to a project

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
INNER JOIN Project P ON E.EmpID = P.EmpID;
```

(b) LEFT OUTER JOIN — all employees, with project if any

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
LEFT OUTER JOIN Project P ON E.EmpID = P.EmpID;
```

(c) RIGHT OUTER JOIN — all projects, with employee if assigned

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
RIGHT OUTER JOIN Project P ON E.EmpID = P.EmpID;
```

(d) FULL OUTER JOIN — every employee and every project

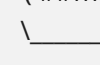
```
SELECT E.EmpName, P.ProjectName
FROM Employee E
FULL OUTER JOIN Project P ON E.EmpID = P.EmpID;
```

(e) CROSS JOIN — Cartesian product of both tables

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
```

CROSS JOIN Project P;

Join coverage diagram

INNER JOIN	LEFT JOIN	RIGHT JOIN	FULL JOIN
$(E \cap P)$	$(E \supseteq \text{match})$	$(P \supseteq \text{match})$	$(E \cup P)$
			
$(E \cap P)$	$(\#\# \text{match})$	$(\text{match} \#\#)$	$(\#\# \text{match} \#)$
			

(keeps only matches
plus all unmatched
rows from both sides)

Q6. Create View 'DeptSalaryView' [5 Marks]

```
CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
       COUNT(E.EmpID) AS TotalEmployees,
       AVG(E.Salary) AS AvgSalary
FROM   Department D
LEFT JOIN Employee E ON D.DeptID = E.DeptID
GROUP BY D.DeptName;
```

Query on the view — departments with average salary above 60000

```
SELECT *
FROM   DeptSalaryView
WHERE  AvgSalary > 60000
ORDER BY AvgSalary DESC;
```

A view stores no data of its own; it is a saved SELECT statement that is re-executed each time it is queried, so DeptSalaryView always reflects the current contents of Employee and Department.

Q7. Relational Algebra — Division: Students Enrolled in ALL Courses [5 Marks]

Given ENROLLED(SID, CourseID) and COURSE(CourseID, CourseName), the division operator ($\div$) is the natural tool: it returns the SID values whose set of associated CourseIDs is a superset of every CourseID offered.

$$\pi \text{ SID, CourseID (ENROLLED)} \div \pi \text{ CourseID (COURSE)}$$

Equivalent expansion (division built from primitive operators)

```
Let R =  $\pi \text{ SID, CourseID (ENROLLED)}$ 
    S =  $\pi \text{ CourseID (COURSE)}$ 
```


$$R \div S = \pi_{SID}(R) - \pi_{SID}((\pi_{SID}(R) \times S) - R)$$

Interpretation diagram

$\pi_{SID}(\text{ENROLLED})$
 |
 $(R \div S) \rightarrow$ SIDs present in R for EVERY course in S
 |
 remove any SID missing at least one course
 |
 Result: Names of students enrolled in all courses
 Final step: $\pi_{Name}(\text{STUDENT} \bowtie (R \div S))$

Q8. SQL DML with Integrity Constraint Enforcement [5 Marks]

Scenario: inserting a new employee, updating a salary, and deleting an employee, on the EMPLOYEE(EmpID PK, EmpName, Salary, DeptID FK) table from Q1/Q2.

(a) INSERT — enforces PRIMARY KEY and FOREIGN KEY constraints

```
INSERT INTO Employee (EmpID, EmpName, Salary, DeptID)
VALUES (101, 'Ananya Rao', 55000, 'D01');
```

- Fails with a PK-violation error if EmpID 101 already exists.
- Fails with an FK-violation error if DeptID 'D01' is not in Department.

(b) UPDATE — enforces CHECK / NOT NULL constraints

```
UPDATE Employee
SET Salary = Salary * 1.10
WHERE DeptID = 'D01';
```

- Fails if a CHECK constraint such as CHECK (Salary > 0) would be violated.

(c) DELETE — enforces referential integrity

```
DELETE FROM Employee
WHERE EmpID = 101;
```

- If Employee is referenced by Project.EmpID with a RESTRICT policy,
- this delete is rejected until the dependent Project rows are removed
- or reassigned. With ON DELETE CASCADE on that FK it would instead
- delete matching Project rows automatically.

Q9. Tuple Relational Calculus — IT Employees Earning > 50000 [5 Marks]

$$\{t \mid t \in \text{EMPLOYEE} \wedge t.\text{DeptID} = \text{'IT'} \wedge t.\text{Salary} > 50000\}$$

Read as: "the set of all tuples t such that t belongs to EMPLOYEE, t 's DeptID equals 'IT', and t 's Salary is greater than 50000." If the department is stored only by name in a separate relation, the calculus expression instead needs an existential quantifier joining across relations:

$$\{t \mid t \in \text{EMPLOYEE} \wedge t.\text{Salary} > 50000 \wedge \exists d (d \in \text{DEPARTMENT} \wedge d.\text{DeptID} = t.\text{DeptID} \wedge d.\text{DeptName} = \text{'IT'})\}$$

Q10. Airline Reservation System — Schema Design & 5 SQL Queries [5 Marks]

Relational schema (primary keys underlined conceptually via PK, foreign keys marked FK):

PASSENGER

Column	Data Type	Constraint
PassengerID	INT	Primary Key
Name	VARCHAR(50)	Not Null
Phone	VARCHAR(15)	Unique
Email	VARCHAR(50)	Unique

FLIGHT

Column	Data Type	Constraint
FlightID	INT	Primary Key
FlightNo	VARCHAR(10)	Unique, Not Null
Source	VARCHAR(30)	Not Null
Destination	VARCHAR(30)	Not Null
DepartureTime	DATETIME	Not Null
Fare	DECIMAL(10,2)	Not Null, CHECK (Fare > 0)

AIRCRAFT

Column	Data Type	Constraint
AircraftID	INT	Primary Key
Model	VARCHAR(30)	Not Null
Capacity	INT	CHECK (Capacity > 0)

RESERVATION

Column	Data Type	Constraint
ReservationID	INT	Primary Key

Column	Data Type	Constraint
PassengerID	INT	Foreign Key → Passenger(PassengerID)
FlightID	INT	Foreign Key → Flight(FlightID)
SeatNo	VARCHAR(5)	Not Null
BookingDate	DATE	Not Null, Default CURRENT_DATE
Status	VARCHAR(10)	CHECK (Status IN ('CONFIRMED', 'CANCELLED', 'WAITLIST'))

Entity relationship diagram

PASSENGER (1) —< RESERVATION >— (1) FLIGHT —> (1) AIRCRAFT
 books many operated by

PASSENGER 1—∞ RESERVATION ∞—1 FLIGHT FLIGHT ∞—1 AIRCRAFT

Query 1 — JOIN: passenger name with their flight details

```
SELECT P.Name, F.FlightNo, F.Source, F.Destination, R.SeatNo
FROM   Reservation R
JOIN   Passenger P ON R.PassengerID = P.PassengerID
JOIN   Flight F   ON R.FlightID   = F.FlightID
WHERE  R.Status = 'CONFIRMED';
```

Query 2 — JOIN with AIRCRAFT: flights and the aircraft model flown

```
SELECT F.FlightNo, F.Source, F.Destination, A.Model, A.Capacity
FROM   Flight F
JOIN   Aircraft A ON F.AircraftID = A.AircraftID
ORDER BY F.DepartureTime;
```

Query 3 — Sub-query: passengers who have never booked a flight

```
SELECT Name
FROM   Passenger
WHERE  PassengerID NOT IN (
        SELECT PassengerID FROM Reservation
      );
```

Query 4 — Sub-query: flights priced above the average fare

```
SELECT FlightNo, Fare
FROM   Flight
WHERE  Fare > ( SELECT AVG(Fare) FROM Flight );
```

Query 5 — View: seats booked per flight, and a query on it

```
CREATE VIEW FlightBookingSummary AS
SELECT F.FlightNo,
       COUNT(R.ReservationID) AS SeatsBooked,
       A.Capacity - COUNT(R.ReservationID) AS SeatsAvailable
FROM   Flight F
JOIN   Aircraft A ON F.AircraftID = A.AircraftID
LEFT JOIN Reservation R
      ON F.FlightID = R.FlightID AND R.Status = 'CONFIRMED'
GROUP BY F.FlightNo, A.Capacity;

-- Query on the view: flights that are more than 90% full
SELECT *
FROM   FlightBookingSummary
WHERE  SeatsBooked > 0.9 * (SeatsBooked + SeatsAvailable);
```